

Quantized DLMs Evaluation

Ahmad Fraij

Outline

- Issues with direct comparison of Llada/Dream Results with Autoregressive Models in SINQ paper
- Evaluation code for Llada and SINQ for perplexity and accuracy based benchmarks
- Evaluation results on some benchmarks

SINQ Evaluation Function

- The evaluate_model function in the quant_model_eval.py in SINQ repo does not work for llada/dream models.
- The HFLM wrapper and the code does not take into account the bidirectional attention mechanism of DLMs and their denoising mechanism

```
lm = HFLM(pretrained=model, tokenizer=tokenizer, batch_size=batch_size)
```

```
for i in tqdm(range(nsamples)):
    batch = testenc[:, (i * lm.seqlen) : ((i + 1) * lm.seqlen)].to(lm.device)
    outputs = lm.model.model(batch)
    hidden_states = outputs[0] # .to(lm.model.lm_head.weight.device)
    logits = lm.model.lm_head(hidden_states)
    shift_logits = logits[:, :-1, :] # .contiguous()
    shift_labels = testenc[:, (i * lm.seqlen) : ((i + 1) * lm.seqlen)][:, 1:].to(lm.device)
    loss_fct = nn.CrossEntropyLoss()
    loss = loss_fct(shift_logits.view(-1, shift_logits.size(-1)), shift_labels.view(-1))
    neg_log_likelihood = loss.float() * lm.seqlen
    nlls.append(neg_log_likelihood)
```

Llada Model Perplexity Evaluation

- Splits tokens into non-overlapping chunks of size=max_length
- Performs Monte Carlo iterations using forward_process + get_logits functions, and computes cross-entropy loss weighted by the mask probability

```
def loglikelihood_rolling(self, requests):
    out = []
    prefix = torch.tensor([], dtype=torch.long)
    for req in tqdm(requests, desc="Computing rolling likelihood..."):
        text = req.args[0]
        tokens = self.tokenizer.encode(text, add_special_tokens=False)
        tokens = torch.tensor(tokens)

        # Non-overlapping chunks of max_length
        nchunks = len(tokens) // self.max_length
        total_ll = 0.0
        for i in range(nchunks):
            chunk = tokens[i * self.max_length : (i + 1) * self.max_length]
            total_ll += self.get_loglikelihood(prefix, chunk)

        out.append(total_ll)
    return out
```

```
@torch.no_grad()
def get_loglikelihood(self, prefix, target):
    seq = torch.concatenate([prefix, target])[None, :]
    seq = seq.repeat((self.batch_size, 1)).to(self.device)

    prompt_index = torch.arange(seq.shape[1], device=self.device) < len(prefix)

    loss_acc = []
    for _ in range(self.mc_num // self.batch_size):
        perturbed_seq, p_mask = self._forward_process(seq, prompt_index)

        mask_indices = perturbed_seq == self.mask_id

        logits = self.get_logits(perturbed_seq, prompt_index)

        loss = F.cross_entropy(logits[mask_indices], seq[mask_indices], reduction='none') / p_mask[mask_indices]
        loss = loss.sum() / self.batch_size
        loss_acc.append(loss.item())

    return - sum(loss_acc) / len(loss_acc)
```

$$\mathcal{L}(\theta) \triangleq -\mathbb{E}_{t, x_0, x_t} \left[\frac{1}{t} \sum_{i=1}^L \mathbf{1}[x_t^i = \text{M}] \log p_{\theta}(x_0^i | x_t) \right]$$

Dream Model Perplexity Evaluation

- Estimates the masked diffusion ELBO via Monte Carlo sampling.
- Forward process samples a random mask rate $t \in (0,1)$ per chunk
- Replace target tokens with [MASK] via Bernoulli sampling.
- Computes weighted cross-entropy loss using importance reweighting.

```
def loglikelihood_rolling(self, requests: List[Instance]) -> List[float]:
    out = []
    prefix = torch.tensor([], dtype=torch.long)
    for req in tqdm(requests, desc="Computing rolling likelihood..."):
        text = req.args[0]
        tokens = self.tokenizer.encode(text, add_special_tokens=False)
        tokens = torch.tensor(tokens)
        nchunks = len(tokens) // self.max_length
        total_ll = 0.0
        for i in range(nchunks):
            chunk = tokens[i * self.max_length:(i + 1) * self.max_length]
            total_ll += -self.eval_target_nll_mc(prefix, chunk)
        out.append(total_ll)
    return out
```

$$L(\theta) = -\mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}), t \sim \mathcal{U}(0,1), \mathbf{x}_t \sim q(\mathbf{x}_t | \mathbf{x}_0)} \sum_{n=1}^N \mathbf{1}_{[\mathbf{x}_t^n = \text{MASK}]} w(t, \mathbf{x}_t, n) \log p_{\theta}(\mathbf{x}_0^n | \mathbf{x}_t),$$

Evaluation On Perplexity Based Wiki2Text

Hyperparameter	LLaDA	Dream
Batch size	1	1
Monte Carlo samples	1	1
Diffusion steps / Steps	1024	1024
Generation length / Max new tokens	1024	128
Block length	1024	–
Remasking strategy	low_confidence	–
Few-shot examples	0	0

Method	Qwen3-1.7B			Qwen3-14B			Qwen3-32B			Dream-7B			LLaDA-8B		
	Mem	Wiki2 ↓	C4 ↓	Mem	Wiki2 ↓	C4 ↓	Mem	Wiki2 ↓	C4 ↓	Mem	Wiki2 ↓	C4 ↓	Mem	Wiki2 ↓	C4 ↓
Original (BF16)	3.44	16.67	19.21	29.54	8.64	12.01	65.52	7.60	10.77	15	11.00	–	15	8.90	–
SINQ (4-bit)	1.42	17.14	19.83	10.56	8.76	12.21	20.73	7.74	10.96	5.1	11.54	–	4.6	9.13	–

Evaluation against other Quantization Techniques

Table 1: Model performance on general tasks under weight-only quantization.

Model	Setting	Method	WinoGrande	PIQA	ARC-C	ARC-E	Hellaswag	MMLU 5-shot	Avg	Drop
LLaDA-8B	FP Model	-	69.9	74.6	46.4	71.1	70.7	65.7	65.5	-
	W4A16 g128	GPTQ	69.7	73.9	47.9	72.5	70.4	64.7	65.3	↓ 0.3%
		AWQ	67.3	70.3	44.5	73.4	68.4	65.6	63.2	↓ 3.5%
	W3A16 g128	GPTQ	67.2	73.3	45.7	71.1	68.8	63.5	63.7	↓ 2.7%
		AWQ	66.4	69.2	42.8	71.8	66.4	64.0	61.8	↓ 5.6%
	FP Model	-	70.2	71.3	54.3	75.9	68.6	64.0	65.7	-
LLaDA-8B -Instruct	W4A16 g128	GPTQ	69.2	74.2	54.8	77.5	68.3	63.4	66.0	↑ 0.3%
		AWQ	68.8	71.0	53.3	76.1	68.1	63.4	64.9	↓ 0.1%
	W3A16 g128	GPTQ	67.4	73.7	50.7	76.4	66.7	62.1	64.1	↓ 2.4%
		AWQ	66.3	69.8	50.5	74.7	66.4	62.3	63.1	↓ 4.0%

Table 5: Evaluation of weight-only quantized Dream-7B on general tasks.

Model	Setting	Method	WinoGrande	PIQA	ARC-C	ARC-E	Avg	Drop
Dream-7B	FP Model	-	68.4	74.4	59.0	83.1	71.2	-
	W4A16 g128	GPTQ	68.2	73.9	58.1	82.1	70.6	↓ 0.8%
		AWQ	65.2	69.6	55.8	82.0	68.2	↓ 4.3%
	W3A16 g128	GPTQ	63.3	69.6	49.9	73.4	64.1	↓ 10.1%
		AWQ	62.8	67.7	50.6	74.5	63.9	↓ 10.3%
	FP Model	-	68.4	74.4	59.0	83.1	71.2	-

Plan for Next Week

- Aim to make SINQ aware of diffusion steps and understand how it can be modified to achieve that
- Analyze how quantization and/or reconstruction errors accumulate across diffusion steps
- Suggest possible modifications to SINQ architecture to reduce reconstruction error
- Run more benchmarks on SINQ quantized Dream and Llada